기말 프로젝트 기획안

세컨드 브레인 플랫폼 (가칭)

AI가 내 지식을 기억하고, 같은 실수를 반복하지 않게 돕는 개인 지식 플랫폼

문서 구분	기말 프로젝트 기획안
버전	v1.1 (교수님 피드백 반영)
작성일	2026년 11월 5일
작성자	김민상 (대림대학교 3학년 2반)
수신	양윤모 교수님
프로젝트 형태	1인 프로젝트

목차

0. 요약

1. 배경과 목표

2. 핵심 기능

3. 서비스 구조: 로컬 우선

4. 수익 모델

5. 경쟁 서비스와 차별점

6. 개발 범위와 일정

7. 역할 분담

8. 위험 요소와 대응

참고 자료

0. 요약

> **한눈에 보기**
>
> - **무엇을**: Claude Code·Codex 같은 AI 도구와 연동하면 대화와 지식이 자동으로 쌓이고 정리되는 세컨드 브레인 플랫폼 (데스크톱 앱)
> - **왜**: AI 도구로 직접 세컨드 브레인을 만들 수는 있지만, 설정이 어렵고 번거로워 일반 사용자는 쓰기 어렵다
> - **어떻게**: 지식은 사용자 기기에 md 파일로 저장한다. 연동한 AI는 우리 서버를 거치지 않고 명령 한 줄로 지식을 찾는다. 서버는 로그인·결제·앱 안 AI 질문만 맡는다
> - **수익**: 무료 요금제 + 월 구독(Pro)
> - **기말 범위**: macOS 데스크톱 MVP — 노트, 뇌 그래프, Claude Code·Codex 연동, AI 질문, 검증되는 실수 노트, 할 일, 요금제 화면 (웹은 체험판)

> 용어 설명
>
> - **세컨드 브레인(Second Brain)**: 머릿속 대신 외부 시스템에 지식을 저장·정리해 두고 필요할 때 꺼내 쓰는 방식
> - **MVP(최소 기능 제품)**: 핵심 가치를 확인할 수 있는 최소한의 기능만 담은 첫 버전

1. 배경과 목표

1.1 문제 정의

문제	현재 상황
지식이 흩어진다	AI 대화, 메모, 회의 내용이 도구마다 흩어져 있어 다시 찾기 어렵다
AI가 같은 실수를 반복한다	새 대화를 시작하면 이전에 바로잡아 준 내용을 기억하지 못한다
해결 방법이 어렵다	Claude Code·Codex의 메모리 파일, 훅, MCP를 설정하면 해결할 수 있지만 개발 지식과 시간이 필요하다

> 용어 설명
>
> - **훅(Hook)**: AI 도구에서 대화 시작·종료 같은 특정 시점에 자동으로 실행되는 작업
> - **MCP(Model Context Protocol)**: AI가 외부 서비스의 데이터와 기능을 쓸 수 있게 연결하는 표준 규격

1.2 해결 방향

번거로운 설정을 플랫폼이 대신한다. 사용자가 연동 버튼을 누르면 플랫폼이 AI 도구에 **온보딩 파일**(스킬·훅)을 설치한다. 그러면 AI는 따로 설명하지 않아도 내 지식을 찾아 쓰고, 대화는 자동으로 지식이 된다.

1.3 대상 사용자와 목표

- **대상 사용자**: AI 도구를 매일 쓰는 대학생·직장인·개발자. 특히 개발 설정 없이 AI를 많이 쓰는 사용자
- **목표 1**: 가입 후 5분 안에 첫 지식 그래프를 만든다
- **목표 2**: 한 번 교정한 실수가 다음 관련 질문에 반영된다 (시연으로 검증)
- **목표 3**: 연동한 AI가 명령 한 줄(10토큰 안팎)로 필요한 지식만 찾게 해 토큰 사용을 줄인다

2. 핵심 기능

No.	기능	설명	무료	Pro
1	**지식 수집**	노트 작성, AI 대화기록·세션 기록 가져오기	○ (개수 제한)	○
2	**뇌 그래프**	저장된 지식을 신경망처럼 연결해 보여 준다	○	○
3	**위키 기반 AI 답변**	앱 안에서 질문하면 관련 지식만 찾아 AI에 함께 보낸다	월 사용량 제한	넉넉한 사용량
4	**AI 실수 노트**	교정 반응을 검증 에이전트가 판단해 규칙으로 저장하고, 관련 질문에 자동 반영한다	직접 교정	자동 감지 포함
5	**문서 정리·할 일**	대화에서 할 일을 뽑아 목록으로 관리한다	직접 등록	자동 추출
6	**자동 최적화**	중복 합치기, 자동 연결, 목차 파일 갱신	[정리하기] 버튼	매일 자동
7	**AI 도구 연동**	Claude Code·Codex에 연동 스킬을 설치해, AI가 내 지식을 바로 찾고 대화를 저장한다	-	○
8	**회의 기록***	음성을 텍스트로 바꾸고 요약해 지식으로 저장	텍스트 붙여넣기	녹음 변환
9	**기기 간 동기화***	암호화된 상태로 여러 기기에서 이어 쓰기	-	○

* 표시는 2단계(기말 이후) 기능이다.

2.1 AI 실수 노트와 검증 에이전트

1. **반응 감지**: 사용자가 [틀렸어]를 누르거나 "아니야", "다시 해" 같은 교정 반응을 보이면 검증을 시작한다
2. **검증**: 검증 에이전트가 진짜 교정인지 판단한다. 사용자 반응을 그대로 믿지 않고, 근거 노트와 교정 내용이 맞는지 확인한다
3. **규칙화**: 교정이 맞으면 "언제 → 무엇을 → 왜" 형태의 규칙으로 만든다. 비슷한 규칙이 있으면 새로 만들지 않고 합친다
4. **적용 검사**: 규칙이 들어간 답변은 내보내기 전에 규칙을 지켰는지 검사하고, 어겼으면 한 번 다시 쓴다
5. **자기 개선(RSI)**: 검증 에이전트가 놓친 실수를 사용자가 나중에 바로잡으면, 그 내용을 검증 에이전트의 검사 기준에 추가한다

점수를 미리 정해 두고 반응에 따라 더하고 빼는 방식은 쓰지 않는다. 사용자는 대충 "좋아"라고 답하는 경우가 많아 점수가 올라가기만 하기 때문이다. 규칙 문장 형식("언제 → 무엇을 → 왜")만 오픈소스 agent-librarian의 rule-creator를 참고했다.

> 용어 설명
>
> - **검증 에이전트**: 답변과 교정을 따로 검사하는 보조 AI
> - **RSI(Recursive Self-Improvement, 재귀적 자기 개선)**: AI가 자기 결과를 검사·수정하고, 그 경험으로 다음 결과를 개선하는 과정을 반복하는 방식. 본 서비스는 AI 모델 자체가 아니라 AI가 쓰는 규칙과 검사 기준을 개선하는 범위에서 적용한다

2.2 신규 사용자 자동 최적화

- **가입 직후**: 가져온 대화기록을 주제별로 묶고, 묶음마다 이름을 붙여 그래프를 자동으로 만든다
- **온보딩 요약**: 사용자의 주요 주제와 자주 받은 교정을 짧게 요약해, 연동한 AI가 첫 대화부터 사용자를 알고 시작하게 한다
- **쓰는 동안**: 중복 합치기, 새 노트 자동 연결, 목차 파일 갱신, 오래 쓰지 않은 지식 보관
- **안전장치**: AI는 지우지 않고 제안만 한다. 자동으로 적용한 변경은 이력을 남기고 되돌릴 수 있다

3. 서비스 구조: 로컬 우선

핵심 원칙은 **"지식은 사용자 기기에, 서버에는 꼭 필요한 정보만"**이다. 연동한 AI는 우리 서버를 거치지 않고 기기의 지식을 바로 찾는다.

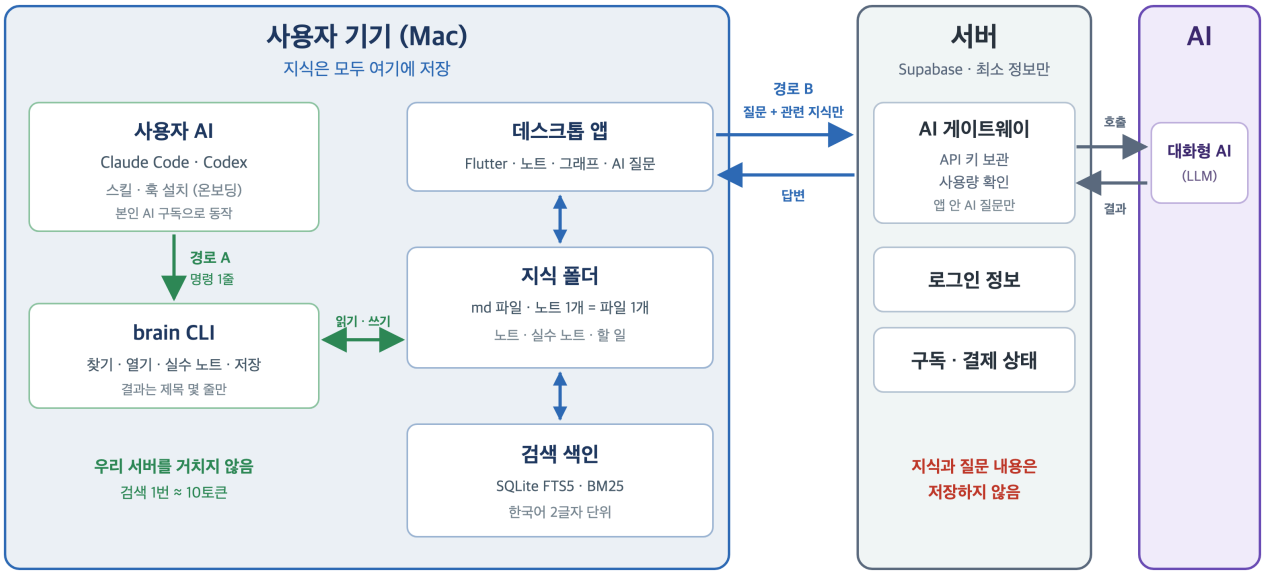


그림 1. 서비스 구조 — 연동한 AI는 서버 없이 기기의 지식을 찾고(경로 A), 서버는 로그인·결제·앱 안 AI 질문(경로 B)만 맡는다

3.1 데이터 저장 위치

데이터	저장 위치	이유
노트·그래프·실수 노트·할 일	**사용자 기기** (md 파일)	개인 지식을 외부에 두지 않고, 사용자 AI와 다른 노트 앱에서도 바로 읽을 수 있다
검색 색인	**사용자 기기** (SQLite)	md 파일에서 언제든 다시 만들 수 있다
로그인 정보	서버	계정 보호
구독·결제 상태	서버	월 결제 갱신 관리
AI 서비스 키(API 키)	서버	사용자 화면에 키가 노출되지 않게 한다
앱 안 AI 질문 내용	**저장하지 않음** (서버는 전달만)	AI 호출에 키가 필요해 서버를 거치지만, 내용은 남기지 않는다
연동한 AI가 찾은 지식	**서버를 거치지 않음**	사용자 AI가 기기에서 직접 찾고, 사용자 본인의 AI 서비스로만 전달된다

3.2 저장 방식

- 저장 위치를 묻는 확인 창 없이 **자동으로 저장**한다. 사용 경험은 일반 클라우드 서비스와 같다
- 데스크톱 앱은 앱 전용 폴더에 **노트 1개를 md 파일 1개로** 저장한다. 웹은 샘플 데이터로 기능을 보여 주는 체험판이다
- 저장 위치와 방식은 설정 화면과 개인정보처리방침에 명시하고, 폴더째 백업할 수 있게 한다

3.3 md 파일이 많아질 때

- **AI가 전부 읽지 않는다**: 색인으로 관련 노트 제목 5개만 찾고, 본문은 필요한 것만 연다
- **목차로 좁힌다**: 맨 위 목차는 주제 목록만 담고, 주제별 목차로 한 단계씩 좁힌다

- **원본과 정리본을 나눈다**: 가져온 원본(raw)은 따로 보관하고, 평소 검색은 정리본(wiki)에서만 한다
- **파일 수를 줄인다**: 자동 정리가 중복 노트를 하나로 합친다
- **측정한다**: 노트 1만 개 기준으로 검색 시간을 측정해 확인한다

> **왜 이 구조인가**
>
> - **대안 A — 모든 지식을 서버에 저장**: 여러 기기에서 쓰기 쉽지만 개인정보 부담과 서버 비용이 크다 → 미채택
> - **대안 B — 임베딩(의미) 검색**: 단어가 달라도 찾지만 노트를 외부 임베딩 서비스로 보내야 한다. 개인 규모에서는 원문 검색으로 충분하다 → 2단계 선택 기능
> - **대안 C — MCP로 연결**: 표준 방식이지만 연결해 두면 도구 설명이 대화에 계속 실려 토큰을 쓴다 → CLI 채택, MCP는 2단계
> - **대안 D — 웹 앱을 본 제품으로**: 설치가 필요 없지만 브라우저 저장소(IndexedDB)가 느리고 사용자 AI가 읽을 수 없다 → 웹은 체험판
> - **채택안 — 데스크톱 앱 + md 파일 + CLI 연동 + 서버 최소 정보**: 개인정보 보호, 간편함, 적은 토큰을 함께 얻는다

3.4 기술 스택

구분	기술	하는 일
화면	Flutter (macOS 앱, 웹 체험판)	로그인, 노트, 뇌 그래프, AI 질문, 할 일, 연동 화면
지식 저장	md 파일 (앱 전용 폴더)	노트·실수 노트·할 일 보관. 사용자 AI와 Obsidian에서도 읽힌다
검색 색인	SQLite FTS5 (BM25)	원문 검색. 한국어는 조사 때문에 2글자 단위로 색인
연동	brain CLI, 연동 스킬·훅	Claude Code·Codex가 명령 한 줄로 내 지식을 찾고 저장
회원·결제	Supabase (Auth, Database)	로그인, 구독 상태 관리 (지식은 저장하지 않음)
AI 게이트웨이	Supabase Edge Functions	앱 안 AI 질문일 때만 AI 키를 숨긴 채 호출하고, 회원 등급과 사용량을 확인
AI	대화형 AI (LLM)	답변, 요약, 교정 검증
결제	모의 결제 화면	기말에서는 화면만 구현, 실서비스 시 결제 대행사 연동

3.5 지식을 찾는 두 경로

단계	경로 A: 연동한 AI	경로 B: 앱 안 AI 질문
질문	평소처럼 Claude Code·Codex에 질문한다	앱 화면에서 질문한다
찾기	AI가 "brain find 검색어"를 실행해 관련 노트 제목 5줄을 받는다	앱이 기기 색인에서 관련 노트 5개와 실수 노트 최대 3개를 찾는다
읽기	필요한 노트만 "brain show"로 연다	찾은 지식만 질문과 함께 AI 게이트웨이로 보낸다
답변	사용자 본인의 AI가 답한다	서버가 등급·사용량을 확인하고 AI에 전달해, 결과만 돌려준다
서버	**거치지 않음**	내용은 저장하지 않음
비용	**우리 비용 없음** (검색 명령 10토큰 안팎)	사용량 1회 차감

예: AI가 "brain find 결제 오류"를 실행하면 "n_0042 결제 웹훅 재시도 규칙"처럼 제목 몇 줄만 받고, 필요할 때 "brain show n_0042"로 본문을 연다.

> 용어 설명
>
> - **md(Markdown) 파일**: 제목·목록 같은 간단한 표시를 넣을 수 있는 텍스트 파일
> - **색인 / BM25**: 색인은 단어마다 그 단어가 든 문서를 미리 정리해 둔 목록이다. BM25는 검색어가 많이 나오고 드문 단어일수록 점수를 높이는 표준 검색 공식이다
> - **CLI**: 글자로 명령을 입력해 쓰는 프로그램. Claude Code·Codex는 CLI 명령을 직접 실행할 수 있다
> - **스킬(Skill) / 온보딩 파일**: 스킬은 AI 도구가 필요할 때 꺼내 읽는 사용 설명서 파일이다. 온보딩 파일은 AI가 처음 읽고 바로 일할 수 있게 하는 안내 파일(스킬·훅 설정)이다
> - **AI 게이트웨이**: AI 키를 보관하고 회원 등급·사용량을 확인한 뒤 AI를 호출해 주는 서버 기능 (Supabase Edge Functions로 구현)

4. 수익 모델

원칙: 비용이 드는 기능(앱 안 AI 호출, 음성 변환, 동기화)은 유료로 제공한다. 지식은 사용자 기기에 저장되고 연동한 AI의 검색은 우리 서버를 거치지 않아 비용이 들지 않으므로, 저장은 무료로 넉넉하게 제공한다.

항목	무료	Pro (월 9,900원, 가안)
노트 저장	최대 200개	무제한
앱 안 AI 질문	월 50회	월 1,000회
AI 실수 노트	직접 교정, 최대 20개	자동 감지 포함, 무제한
자동 정리	[정리하기] 버튼	매일 자동
AI 도구 연동	-	Claude Code·Codex
회의 기록	텍스트 붙여넣기	녹음 변환
기기 간 동기화	-	암호화 동기화

금액과 수치는 가안이며, 개발 후 실제 AI 비용을 측정해 조정한다. AI 질문 횟수는 앱 안 질문에만 적용하고, 연동한 AI의 검색은 횟수에 포함하지 않는다.

참고 사례: Obsidian은 앱을 개인 사용자에게 무료로 제공해 기기에 저장하게 하고, 기기 간 동기화를 유료로 판매한다. 본 서비스도 같은 구조를 따른다.

5. 경쟁 서비스와 차별점

구분	예시	특징	본 서비스와의 차이
노트 도구	Notion, Obsidian	사람이 직접 쓰고 정리한다	AI가 지식을 자동으로 모으고 정리한다
AI 도구 자체 기능	Claude Code 메모리, Codex AGENTS.md	도구마다 따로 저장되고, 사용자가 직접 관리한다	Claude·Codex가 같은 지식을 함께 쓰고, 검증된 규칙만 남긴다
AI 메모리 오픈소스	claude-mem, mem0	개발자가 직접 설치·설정해 쓴다	설정 없이 가입·연동만으로 쓴다
회의 기록 서비스	클로바노트	회의 녹음과 요약에 집중한다	회의 내용을 기존 지식·할 일과 연결한다

차별점 4가지

1. **연동만으로 자동**: 연동 버튼이 온보딩 파일(스킬·훅)을 설치해 설정 없이 시작한다
2. **검증되는 실수 노트**: 검증 에이전트가 맞는 규칙만 남기고, 놓친 실수로 검사 기준을 스스로 고친다

3. **도구 공통 + 로컬 우선**: Claude와 Codex가 내 기기의 같은 지식을 함께 쓰고, 지식은 내 기기에 남는다
4. **적은 토큰**: 지식을 찾는 명령이 10토큰 안팎이고, 필요한 노트만 열어 AI 사용량을 아낀다

6. 개발 범위와 일정

6.1 기말 범위 (MVP)

구분	기능
포함	회원가입·로그인, macOS 데스크톱 앱, 노트 md 저장과 검색 색인, AI 대화기록 파일 가져오기, 뇌 그래프, brain CLI와 Claude Code·Codex 연동 스킬, 앱 안 AI 질문, 실수 노트와 검증 에이전트, 할 일, 요금제·모의 결제 화면, 웹 체험판
제외 (2단계)	임베딩 검색, MCP 연결, Windows 앱(Windows PC 필요), 회의 녹음 변환, 매일 자동 정리, 암호화 동기화, 실제 결제

범위를 정한 이유: 핵심 가치인 "지식 저장 → 연동한 AI가 찾아 쓰기 → 실수 없는 답변" 흐름을 기말 안에 완성해 시연하기 위함이다. 임베딩 검색을 원문 검색으로 바꿔 줄어든 작업량만큼 연동 기능을 기말 범위로 당겼다.

6.2 일정 (12월 8일 제출)

단계	기간	작업	산출물
1주차	11/06 ~ 11/12	기획 확정, 설계서·화면 설계, 개발 환경(Xcode·Flutter 데스크톱) 구성, 연동 스킬 초안	설계서, 화면 설계, 스킬 초안
2주차	11/13 ~ 11/19	회원가입·로그인, 노트 md 저장, 검색 색인, AI 대화기록 가져오기	로그인·노트·검색 기능
3주차	11/20 ~ 11/26	brain CLI, 연동 스킬·훅 설치, 뇌 그래프, AI 게이트웨이와 앱 안 AI 질문	연동·그래프·AI 질문 기능
4주차	11/27 ~ 12/03	실수 노트와 검증 에이전트, 할 일, 요금제·모의 결제, 웹 체험판	핵심 기능 완성본
마무리	12/04 ~ 12/07	통합 테스트, 버그 수정, 시연 영상, 결과 보고서 (직접 만든 스킬 첨부)	시연 영상·결과 보고서
제출	**12/08**	**최종 결과물 제출**	-

12/04 ~ 12/07은 테스트와 보완을 위한 여유 기간으로 두어, 일정이 밀려도 제출일을 지킨다.

7. 역할 분담

1인 프로젝트이므로 실제 회사의 직무 기준으로 역할을 나누고, 모든 역할을 본인이 수행한다. 역할마다 산출물을 정해 단계별로 스스로 점검한다.

역할	담당 업무	산출물	비중	담당
PM (기획)	요구사항 정의, 일정·범위 관리	기획안, 주간 진행 기록	15%	김민상
UX/UI 디자인	화면 흐름, 화면 설계, 디자인	화면 설계서	15%	김민상
프론트엔드	Flutter 데스크톱 앱, 뇌 그래프, 웹 체험판	데스크톱 앱, 웹 체험판	25%	김민상

역할	담당 업무	산출물	비중	담당
백엔드	로그인, 구독 관리, AI 게이트웨이	서버 기능	15%	김민상
AI·연동	검색 색인, brain CLI, 연동 스킬, 검증 에이전트, 프롬프트 설계	AI·연동 기능, 연동 스킬	20%	김민상
QA·문서	테스트, 버그 관리, 결과 보고서, 발표	테스트 결과, 결과 보고서	10%	김민상

7.1 작업 원칙

- 상세 기술 설계는 별도 내부 설계서로 관리하고, 개발은 설계서를 기준으로 진행한다
- AI 코딩 도구는 보조로만 사용하고, 기능 설계·핵심 로직·검증은 직접 수행한다
- AI가 만든 코드는 이해하고 검토한 뒤에만 반영한다
- 연동 스킬은 직접 만들고, 제작 과정과 시행착오를 기록해 결과 보고서에 참고 자료로 첨부한다
- 참고한 오픈소스는 출처와 라이선스를 문서에 표기한다

8. 위험 요소와 대응

위험	영향	대응
기기 데이터 유실	사용자 지식 유실	md 파일이라 폴더째 백업 가능, 백업 안내, (2단계) 암호화 동기화
md 파일 증가	검색 속도 저하, 토큰 증가	색인으로 관련 노트만 찾기, 목차로 범위 좁히기, 원본 분리, 중복 합치기 (3.3 참고)
AI 비용 증가	운영비 부담	연동 경로는 서버를 거치지 않음, 앱 안 질문은 관련 지식만 전송, 무료 사용량 제한
개인정보·비밀값 노출	서비스 신뢰 하락	서버에 지식 비저장, 전송 전 API 키·비밀번호 자동 가림
AI의 잘못된 학습	엉뚱한 규칙 적용	검증 에이전트가 확인 가능한 기준으로 판단, 규칙 개수 상한, 수정·삭제·되돌리기
개발 환경 준비	개발 착수 지연	1주차에 Xcode 설치와 빈 macOS 앱 빌드부터 확인
일정 지연	기말 미완성	MVP 범위 고정, 늦어지면 웹 체험판 → Codex 연동 순으로 2단계 이월

참고 자료

- Andrej Karpathy, "LLM Wiki" — 원본·정리본 구분과 목차 파일 구조 참고 — gist.github.com/karpathy/442a6bf555914893e9891c11519de94f
- j-token/agent-librarian (rule-creator 스킬, Apache-2.0) — 규칙 문장 형식만 참고 — github.com/j-token/agent-librarian
- SQLite FTS5 공식 문서 — sqlite.org/fts5.html
- Claude Code 공식 문서 (Skills, Subagents, Hooks) — code.claude.com/docs/en/skills
- Codex 공식 문서 (Skills, AGENTS.md, Hooks) — developers.openai.com/codex/skills
- 직접 제작: second-brain 연동 스킬 — 1주차 초안 작성, 결과 보고서에 제작 기록과 함께 첨부